

Những Kiến Thức Cốt Lõi Của Java – Giải Thích Chi Tiết

1. Nền Tảng Ngôn Ngữ

Kiểu dữ liệu

Java có 8 kiểu **nguyên thủy (primitive)**: `byte`, `short`, `int`, `long`, `float`, `double`, `char`, `boolean`. Chúng lưu trực tiếp giá trị trong bộ nhớ Stack, tốc độ nhanh, không tốn overhead của object.

Ngược lại, **kiểu tham chiếu (reference type)** như `String`, mảng, và các object tự định nghĩa — biến chỉ lưu địa chỉ tham chiếu tới object nằm trên Heap.

```
int x = 10;           // primitive
String s = "hello";   // reference
Integer boxed = 10;   // wrapper class của int
```

Autoboxing / Unboxing

Java tự động chuyển đổi giữa kiểu nguyên thủy và wrapper class tương ứng (`int` ↔ `Integer`, `double` ↔ `Double` ...).

```
Integer a = 5;        // autoboxing: int -> Integer
int b = a;            // unboxing: Integer -> int
```

⚠ Lưu ý: so sánh `Integer` bằng `==` có thể sai vì đây là so sánh tham chiếu, không phải giá trị (trừ vùng cache -128 đến 127). Nên dùng `.equals()`.

Cấu trúc điều khiển

- `if/else`, `switch` (Java 14+ hỗ trợ switch expression gọn hơn)
- `for`, `while`, `do-while`, `for-each` (`for (int i : arr)`)

Ép kiểu (Casting)

- Widening (tự động)**: `int` → `long` → `double` (an toàn, không mất dữ liệu)
- Narrowing (phải ép kiểu tường minh)**: `double` → `int` (có thể mất dữ liệu)

```
double d = 9.7;
int i = (int) d;    // i = 9, mất phần thập phân
```

2. Hướng Đối Tượng (OOP)

Đây là phần **quan trọng nhất** vì Java là ngôn ngữ OOP thuần túy nhất trong các ngôn ngữ phổ biến.

Class & Object

- **Class** là bản thiết kế (blueprint), **Object** là thực thể được tạo ra từ class đó (instance).

```
class Dog {
    String name;
    void bark() { System.out.println(name + " sủa!"); }
}

Dog d = new Dog();    // object
d.name = "Lu";
d.bark();
```

4 tính chất của OOP

a) Encapsulation (Đóng gói) Ẩn dữ liệu bên trong class, chỉ cho truy cập qua getter/setter, giúp bảo vệ dữ liệu và kiểm soát cách nó bị thay đổi.

```
class Account {
    private double balance;    // ẩn, không cho truy cập trực tiếp
    public double getBalance() { return balance; }
    public void deposit(double amt) {
        if (amt > 0) balance += amt;
    }
}
```

b) Inheritance (Kế thừa) Một class có thể kế thừa thuộc tính/phương thức từ class khác bằng `extends`, giúp tái sử dụng code.

```
class Animal {
    void eat() { System.out.println("Đang ăn"); }
}
class Cat extends Animal {
    void meow() { System.out.println("Meo"); }
}
```

c) Polymorphism (Đa hình) Cùng một hành động nhưng biểu hiện khác nhau tùy vào object thực thi. - **Compile-time (Overloading)**: cùng tên phương thức, khác tham số - **Runtime (Overriding)**: class con định nghĩa lại phương thức của class cha

```
class Shape {
    double area() { return 0; }
}
class Circle extends Shape {
    double r;
    @Override
    double area() { return Math.PI * r * r; }    // overriding
}

void print(int a, int b) {}    // overloading
void print(double a) {}    // overloading
```

d) Abstraction (Trừu tượng) Chỉ hiển thị các đặc điểm cần thiết, ẩn đi chi tiết triển khai — thực hiện qua `abstract class` hoặc `interface`.

Interface vs Abstract Class

Tiêu chí	Interface	Abstract Class
Đa kế thừa	1 class implement nhiều interface	chỉ extends 1 abstract class
Constructor	Không có	Có thể có
Phương thức	Trước Java 8: chỉ abstract. Từ Java 8+: có thể có <code>default</code> , <code>static</code>	Có thể có cả abstract lẫn concrete method
Khi nào dùng	Định nghĩa “hợp đồng” hành vi (contract), không quan tâm quan	Khi các class con có quan hệ họ hàng rõ ràng và muốn chia sẻ

```
interface Flyable {
    void fly();
}
abstract class Vehicle {
    abstract void move();
    void horn() { System.out.println("Bíp bíp"); } // method cụ thể
}
```

Constructor

Phương thức đặc biệt để khởi tạo object, trùng tên với class, không có kiểu trả về.

```
class Person {
    String name;
    Person(String name) { this.name = name; } // constructor
    Person() { this("Unknown"); } // gọi constructor khác qua this()
}
```

Từ khóa quan trọng

- **static**: thuộc về class, không thuộc về instance cụ thể nào (dùng chung cho mọi object)
- **final**: biến không đổi giá trị, method không bị override, class không bị kế thừa
- **this**: tham chiếu tới object hiện tại
- **super**: tham chiếu tới class cha (gọi constructor cha, method cha)

```
class Base {
    Base() { System.out.println("Base constructor"); }
}
class Derived extends Base {
    Derived() {
        super(); // gọi constructor của Base
        System.out.println("Derived constructor");
    }
}
```

3. Collections Framework

Đây là bộ công cụ để lưu trữ và thao tác với tập hợp dữ liệu.

List — có thứ tự, cho phép trùng lặp

- **ArrayList**: dùng mảng động bên trong, truy cập theo index nhanh ($O(1)$), nhưng chèn/xóa ở giữa chậm ($O(n)$)
- **LinkedList**: dùng danh sách liên kết đôi, chèn/xóa nhanh ($O(1)$) nhưng truy cập theo index chậm ($O(n)$)

```
List<String> list = new ArrayList<>();
list.add("A"); list.add("B");
```

Set — không cho phép trùng lặp

- **HashSet**: không có thứ tự, tốc độ tra cứu nhanh nhất ($O(1)$ trung bình)
- **LinkedHashSet**: giữ thứ tự chèn vào
- **TreeSet**: tự động sắp xếp theo thứ tự (dùng cây đỏ-đen, $O(\log n)$)

Map — lưu cặp key-value

- **HashMap**: nhanh, không đảm bảo thứ tự
- **LinkedHashMap**: giữ thứ tự chèn
- **TreeMap**: tự sắp xếp theo key

```
Map<String, Integer> map = new HashMap<>();
map.put("Táo", 5);
map.get("Táo"); // 5
```

Queue / Deque

- **Queue:** FIFO (vào trước ra trước) — dùng cho hàng đợi xử lý
- **Deque:** có thể thêm/xóa ở cả 2 đầu, dùng làm Stack (LIFO) hoặc Queue

Bảng so sánh Big O nhanh

Cấu trúc	Truy cập	Tìm kiếm	Thêm/Xóa
ArrayList	O(1)	O(n)	O(n)
LinkedList	O(n)	O(n)	O(1)
HashMap	-	O(1)	O(1)
TreeMap	-	O(log n)	O(log n)

Nguyên tắc chọn: cần truy cập ngẫu nhiên nhiều → ArrayList; cần thêm/xóa nhiều ở đầu/giữa → LinkedList; cần tra cứu nhanh theo key → HashMap; cần dữ liệu luôn sắp xếp → TreeMap/TreeSet.

4. Xử Lý Ngoại Lệ (Exception Handling)

try/catch/finally

```
try {
    int result = 10 / 0;
} catch (ArithmeticException e) {
    System.out.println("Lỗi: " + e.getMessage());
} finally {
    System.out.println("Luôn chạy dù có lỗi hay không");
}
```

try-with-resources

Tự động đóng resource (file, connection...) sau khi dùng xong, không cần viết `finally` để close thủ công.

```
try (BufferedReader br = new BufferedReader(new FileReader("file.txt"))) {
    String line = br.readLine();
} catch (IOException e) {
    e.printStackTrace();
}
```

Checked vs Unchecked Exception

- **Checked** (kế thừa `Exception`, không phải `RuntimeException`): bắt buộc phải xử lý hoặc khai báo `throws` — ví dụ `IOException`, `SQLException`
- **Unchecked** (kế thừa `RuntimeException`): không bắt buộc xử lý, thường do lỗi logic — ví dụ `NullPointerException`, `ArrayIndexOutOfBoundsException`

Custom Exception

```
class InsufficientFundsException extends Exception {
    public InsufficientFundsException(String message) {
        super(message);
    }
}

void withdraw(double amount) throws InsufficientFundsException {
```

```

        if (amount > balance) {
            throw new InsufficientFundsException("Số dư không đủ");
        }
    }
}

```

5. Generics

Cho phép viết class/method hoạt động với nhiều kiểu dữ liệu mà vẫn đảm bảo an toàn kiểu (type-safe) tại compile-time, tránh phải ép kiểu thủ công và lỗi runtime.

```

class Box<T> {
    private T value;
    public void set(T value) { this.value = value; }
    public T get() { return value; }
}

Box<String> box = new Box<>();
box.set("Hello");
String s = box.get(); // không cần ép kiểu

```

Wildcard

- ? extends T: chấp nhận T hoặc class con của T (dùng khi chỉ **đọc** dữ liệu — “Producer extends”)
- ? super T: chấp nhận T hoặc class cha của T (dùng khi chỉ **ghi** dữ liệu — “Consumer super”)

```

void printList(List<? extends Number> list) {
    for (Number n : list) System.out.println(n);
}

```

Ứng dụng trong Collections

Toàn bộ Collections Framework đều dùng Generics: `List<String>`, `Map<String, Integer>` ... giúp compiler bắt lỗi kiểu dữ liệu ngay khi biên dịch thay vì để lỗi xảy ra lúc chạy.

6. Java 8+ — Lập Trình Hàm (Functional Programming)

Bắt buộc phải nắm vì hầu hết code Java hiện đại (đặc biệt trong Spring Boot) đều dùng phong cách này.

Lambda Expression

Cách viết gọn cho một function/method ẩn danh.

```

// Trước Java 8
Runnable r1 = new Runnable() {
    public void run() { System.out.println("Chạy"); }
};

// Từ Java 8
Runnable r2 = () -> System.out.println("Chạy");

```

Functional Interfaces

Interface chỉ có **1 phương thức trừu tượng**, dùng làm target cho lambda: - `Function<T, R>`: nhận T, trả về R — `apply()` - `Predicate<T>`: nhận T, trả về boolean — `test()` - `Supplier<T>`: không nhận gì, trả về T — `get()` - `Consumer<T>`: nhận T, không trả về gì — `accept()`

```

Function<Integer, Integer> square = x -> x * x;
Predicate<Integer> isEven = x -> x % 2 == 0;
Consumer<String> printer = s -> System.out.println(s);
Supplier<Double> random = () -> Math.random();

```

Stream API

Xử lý tập hợp dữ liệu theo phong cách khai báo (declarative), thay vì viết vòng lặp thủ công.

```
List<String> names = List.of("An", "Bình", "Cường", "Anh");

List<String> result = names.stream()
    .filter(name -> name.startsWith("A")) // lọc
    .map(String::toUpperCase)             // biến đổi
    .sorted()                             // sắp xếp
    .collect(Collectors.toList());        // gom kết quả

// result = ["AN", "ANH"]
```

Các thao tác chính: - `filter()` : lọc phần tử theo điều kiện - `map()` : biến đổi từng phần tử - `reduce()` : gộp tất cả phần tử thành 1 giá trị - `collect()` : gom kết quả thành List/Set/Map

```
int sum = List.of(1, 2, 3, 4).stream()
    .reduce(0, (a, b) -> a + b); // sum = 10
```

Optional

Bao bọc giá trị có thể null, giúp tránh `NullPointerException` và buộc lập trình viên xử lý trường hợp rỗng tường minh.

```
Optional<String> opt = Optional.ofNullable(getName());
String name = opt.orElse("Không có tên");

opt.ifPresent(n -> System.out.println("Tên: " + n));
```

Method Reference

Viết gọn hơn cho lambda khi chỉ gọi lại 1 method có sẵn.

```
names.forEach(System.out::println); // thay vì names.forEach(n -> System.out.println(n))
```

7. Đa Luồng (Multithreading & Concurrency)

Thread & Runnable

```
// Cách 1: extends Thread
class MyThread extends Thread {
    public void run() { System.out.println("Đang chạy"); }
}

// Cách 2: implements Runnable (khuyến khích hơn vì Java không hỗ trợ đa kế thừa class)
Runnable task = () -> System.out.println("Đang chạy");
new Thread(task).start();
```

Synchronized & Lock

Dùng để tránh **race condition** khi nhiều luồng cùng truy cập một tài nguyên chung.

```
class Counter {
    private int count = 0;
    public synchronized void increment() { // chỉ 1 thread được vào tại 1 thời điểm
        count++;
    }
}
```

`Lock` (từ `java.util.concurrent.locks`) linh hoạt hơn `synchronized`: có thể thử lock có thời hạn

(tryLock), lock có thể ngắt được (lockInterruptibly).

Executor Service & Thread Pool

Thay vì tự tạo Thread thủ công (tốn tài nguyên), dùng pool để tái sử dụng luồng.

```
ExecutorService executor = Executors.newFixedThreadPool(4);
executor.submit(() -> System.out.println("Task chạy trong pool"));
executor.shutdown();
```

Concurrent Collections

Các collection an toàn với đa luồng, hiệu năng tốt hơn việc tự synchronized toàn bộ Map/List: -

ConcurrentHashMap: HashMap an toàn luồng, khóa theo từng phần (segment) thay vì khóa toàn bộ -

CopyOnWriteArrayList: an toàn cho trường hợp đọc nhiều, ghi ít

8. I/O & Serialization

File I/O / Streams

- **InputStream/OutputStream**: xử lý dữ liệu dạng byte (file nhị phân, ảnh...)
- **Reader/Writer**: xử lý dữ liệu dạng ký tự (text, hỗ trợ encoding tốt hơn)

```
try (FileWriter writer = new FileWriter("data.txt")) {
    writer.write("Xin chào");
}

try (BufferedReader reader = new BufferedReader(new FileReader("data.txt"))) {
    String line = reader.readLine();
}
```

Serializable

Cho phép chuyển object thành chuỗi byte để lưu file hoặc truyền qua mạng, và khôi phục lại (deserialize) sau đó.

```
class User implements Serializable {
    String name;
    int age;
}

// Ghi object ra file
ObjectOutputStream oos = new ObjectOutputStream(new FileOutputStream("user.dat"));
oos.writeObject(new User());
```

9. JVM Cơ Bản

Quy trình chạy code Java

1. File .java được **compile** bởi javac thành bytecode .class
2. **JVM** (Java Virtual Machine) đọc bytecode và thực thi — đây là lý do Java có khẩu hiệu “Write Once, Run Anywhere” (chỉ cần có JVM tương ứng cho từng hệ điều hành)
3. JIT Compiler (Just-In-Time) biên dịch bytecode thành mã máy native tại runtime để tăng tốc

Garbage Collection (GC)

JVM tự động thu hồi bộ nhớ của các object không còn được tham chiếu tới nữa, giúp lập trình viên không phải tự quản lý free() như C/C++.

Cơ chế cơ bản: GC quét Heap, xác định object nào không còn “sống” (không còn tham chiếu từ root nào), rồi giải phóng vùng nhớ đó.

Bộ nhớ: Heap vs Stack

- **Stack:** lưu biến local, tham số method, thông tin gọi hàm — mỗi thread có 1 stack riêng, tự giải phóng khi method kết thúc
- **Heap:** lưu object và array — dùng chung cho toàn bộ chương trình, được quản lý bởi Garbage Collector

```
void method() {  
    int x = 5;           // x nằm trên Stack  
    Person p = new Person(); // object Person nằm trên Heap, biến p (tham chiếu) nằm trên Stack  
}
```

10. Công Cụ & Hệ Sinh Thái (nếu học để đi làm)

Build Tools

- **Maven:** quản lý dependency và build project qua file `pom.xml`, theo cấu trúc convention-over-configuration
- **Gradle:** linh hoạt hơn Maven, dùng Groovy/Kotlin DSL, build nhanh hơn nhờ cơ chế cache thông minh

Testing — JUnit

Framework viết unit test tiêu chuẩn của Java.

```
@Test  
void testAdd() {  
    Calculator calc = new Calculator();  
    assertEquals(5, calc.add(2, 3));  
}
```

Spring / Spring Boot

Framework phổ biến nhất cho backend Java hiện nay: - **Dependency Injection:** Spring tự quản lý vòng đời và tiêm các object phụ thuộc thay vì tự `new` thủ công - **Spring Boot:** đơn giản hóa việc cấu hình Spring, tự động cấu hình (auto-configuration), có sẵn embedded server (Tomcat) để chạy ứng dụng ngay mà không cần deploy riêng - Các module phổ biến: Spring MVC (xây REST API), Spring Data JPA (làm việc với database), Spring Security (xác thực/phân quyền)

Lộ Trình Học Đề Xuất

1. **Nền tảng + OOP** (phần 1, 2) — nắm chắc trước khi học gì khác
2. **Collections + Exception Handling** (phần 3, 4) — dùng hàng ngày trong code thực tế
3. **Generics** (phần 5) — hiểu để đọc code Collections thành thạo
4. **Java 8 Stream/Lambda** (phần 6) — bắt buộc cho công việc hiện đại
5. **Multithreading** (phần 7) — nâng cao, cần khi làm hệ thống có hiệu năng cao
6. **I/O & JVM** (phần 8, 9) — hiểu để debug và tối ưu
7. **Công cụ + Spring Boot** (phần 10) — khi sẵn sàng làm dự án thực tế backend